

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»

На правах рукописи

Соколов Павел Павлович

**ПРАКТИЧНЫЕ ЗАВИСИМЫЕ ТИПЫ: ВВЕДЕНИЕ ЛЕГКОВЕСНЫХ
МЕТОДОВ ФОРМАЛЬНОЙ ВЕРИФИКАЦИИ В МЕЙНСТРИМНОЕ
ПРОГРАММИРОВАНИЕ**

ДИССЕРТАЦИЯ
на соискание ученой степени
кандидата компьютерных наук

Научный руководитель:
доктор физико-математических наук,
профессор Кузнецов Степан Львович

Москва — 2027

Оглавление

Введение	4
Актуальность исследования	4
Степень разработанности исследовательской проблемы	4
Объект и предмет исследования	5
Цель и задачи исследования	6
Методология и методы исследования	6
Научная новизна исследования	6
Практическая значимость работы	7
Степень достоверности и апробация результатов	7
1 Чистая система типов с рангами функций в качестве сортов	8
1.1 Мотивация и идея	8
1.2 Чистые системы типов	8
1.2.1 Чистые системы типов: синтаксис	8
1.2.2 Чистые системы типов: правила одношаговой редукции	8
1.2.3 Чистые системы типов: правила типизации	9
1.3 «Комбинаторные» свойства ЧСТ	9
1.3.1 О здравости ЧСТ	9
1.3.2 Парадокс Жирара	10
1.3.3 Как насчёт рангов?	10
1.4 Файл <code>AbstractRewriting.v</code>	10
1.5 Файл <code>Syntax.v</code>	12
1.6 Файл <code>Congruence.v</code>	13
1.7 Файл <code>Reduction.v</code>	14
1.8 Файл <code>Confluence.v</code>	15
1.9 Файл <code>DefEq.v</code>	16
1.10 Файл <code>Context.v</code>	16
1.11 Файл <code>Levels.v</code>	17
1.12 Файл <code>Typing.v</code>	17
1.13 Файл <code>Inversions.v</code>	18
1.14 Файл <code>WF.v</code>	19
1.15 Файл <code>Metatheory.v</code>	19
1.16 Файл <code>Atomic.v</code>	19
1.17 Файл <code>StronglyNormalizing.v</code>	20
1.18 Файл <code>LambdaSet.v</code>	20

2	К системам типов, способных к самореинтерпретации	22
2.1	Введение	22
2.1.1	Вклад	23
2.2	Теорема о невозможности внутренней интерпретации	23
2.3	Почти просто типизированный почти рефлексивный язык: HM-R	25
2.3.1	Синтаксис и семантика HM-R	26
2.3.2	Определение модели STLC внутри HM-R	26
2.3.3	Полиморфность вселенных через переинтерпретацию	27
2.3.4	Параметричность через переинтерпретацию	27
2.4	Свойства HM-R	28
2.4.1	Корректность	28
2.4.2	Консервативность	28
2.4.3	Вывод типов	28
2.5	Рефлексивная теория типов Мартина–Лёфа: MLTT-R	28
2.5.1	Синтаксис	28
2.5.2	Модели MLTT	29
2.5.3	Свойства MLTT-R	29
2.5.4	Применения переинтерпретации	29
2.6	Сравнение с существующими подходами	30
2.6.1	Смещение Макбрайда	30
2.6.2	Внутренняя параметричность	30
2.6.3	Отображённая теория типов	31
2.7	Обсуждение и ограничения	31
2.7.1	Ограничения на модели	31
2.7.2	Вывод типов	31
2.7.3	Возможности для будущих работ	31
2.8	Заключение	31
	Заключение	33
	Основные результаты исследования	33
	Общие выводы	33
	Перспективы дальнейшей разработки	33
	Основные публикации соискателя	33
A	Список сокращений и условных обозначений	34
B	Словарь терминов	35
C	Список литературы	36
D	Список иллюстрированного материала	38
E	Приложения	39

Введение

Актуальность исследования

Подавляющее большинство используемых сегодня языков программирования не подходит для написания корректного по построению кода. В случае же, когда требуется удостовериться в корректном исполнении программы (исполнение без ошибок, соответствие спецификации и так далее), программисту необходимо настраивать дополнительные инструменты: статические анализаторы, автоматическое тестирование, средства проверки модели и так далее. К сожалению, ни одно из используемых средств не решает задачу полностью: какие-то из них не гарантируют корректности¹, другие имеют (сверх)экспоненциальные асимптотики исполнения и не могут быть использованы для полного анализа проверяемых систем². В конце концов, сама необходимость дополнительной настройки, коммерческое лицензирование и сложность использования мешают распространению подобных технологий.

Так, исчерпывающим решением было бы легковесное, бесплатное, удобное для использования, встроенное в стандартную поставку языка программирования средство формальной спецификации и формальной верификации программной системы; по определению, сформулированному Бенджамином К. Пирсом [26], таким средством является *система типов*. Приведём его здесь:

Система типов — это гибко управляемый синтаксический метод доказательства отсутствия в программе определенных видов поведения при помощи классификации выражений языка по разновидностям вычисляемых ими значений (*типам*).

Становится возможным поставить следующую *исследовательскую проблему*: Какой должна быть система типов языка программирования, чтобы

- Она позволяла легко формулировать и легко (в идеале — автоматически) доказывать свойства программ, интересные в мейнстримном программировании?
- Полученным языком было удобно пользоваться широкому кругу пользователей?

Степень разработанности исследовательской проблемы

К подтверждению гипотезы, что теорию типов можно использовать для формальной верификации компьютерных программ, логики и исследователи компьютерных наук приходили постепенно. Пожалуй, первым удачным экспериментом в этой области следует считать Исчисление Конструкций (Calculus of Constructions, CoC) [9] Тьерри Коканда, в

¹Здесь уместно процитировать Эдсгера В. Дейкстру: “... тестирование программы может весьма эффективно продемонстрировать наличие багов, но совершенно неприменимо, чтобы показать их отсутствие” [10].

²Однако, например, проверку моделей вполне успешно используют для верификации критических компонентов системы и рискованных оптимизаций. [19]

котором можно не только реализовывать алгоритмы в виде программ на некотором функциональном языке, но и доказывать их свойства в той же самой системе; практическая реализация этого исчисления сегодня известна как Rosq (ранее Coq) [23].

Эта замечательная особенность CoC возникла в результате принятия парадигмы *утверждения в качестве (некоторых) типов* [25]. В её рамках доказываемое утверждение φ кодируется в виде некоторого *типа* F , представимого в выбранной системе (так что язык типов должен быть достаточно выразительным, чтобы кодировать все утверждения, которые мы хотим доказать). Тогда задача доказательства $\vdash \varphi$ сводится к задаче построения выражения f , имеющего тип F ($\vdash f : F$); поиск такого выражения может производиться как вручную, так и с помощью автоматических средств³.

Однако следует иметь в виду, что Rosq и другие системы интерактивных доказательств в первую очередь являются *системами доказательств*, нежели *языками программирования*: как правило, скрипты для таких систем не запускаются в качестве самостоятельных программ, а только подаются на вход алгоритму проверки типов⁴. Значимыми исключениями являются языки Agda и Lean: в соответствующих компиляторах есть опции, позволяющие компилировать и запускать скрипты на этих языках.

Agda [20], как и язык, на котором она реализована (Haskell), в первую очередь является тестовой площадкой для различных расширений CoC: доступны опции для подключения коиндуктивных типов [2] и ограниченной рекурсии [8], кубических типов [24], логически противоречивого правила Type-in-Type и многих других. К сожалению, это делает ядро системы перегруженным и малоприменимым в критических ситуациях.

Система Lean [17] получила широкую известность ввиду прорывного по тогдашним меркам инструментария и распространения в среде математиков-энтузиастов. Кроме этого, архитектура системы типов и стандартных библиотек была спроектирована с учётом удобства переноса доказательств из классической математики; в решении, избранном создателями Lean, для этого пришлось пожертвовать достаточно важными свойствами базовой системы CoC, в частности свойством *сохранения типизации*: если $e_1 : T$ и $e_1 \rightarrow e_2$ (« e_1 за один шаг вычисляется в e_2 »), то $e_2 : T$. Повторюсь: это свойство в Lean НЕ выполняется. Если быть точным, можно подобрать типизируемый терм e_1 такой, что результат вычисления e_2 не будет типизироваться. Данное обстоятельство, конечно, не делает данную логическую систему противоречивой (в частности, существует теоретико-модельное доказательство непротиворечивости системы Lean [7]), но является серьёзным неудобством при неумелой работе с системой. Более того, если такое базовое синтаксическое свойство не выполнено, возникают сомнения в применимости подобной системы для программирования, т.е. акта дизайна программ для исполнения на переписывающей системе.

Таким образом, можно сделать вывод, что на данный момент не существует *языка программирования* на основе CoC, сохраняющего важные вычислительные свойства и пригодного для разработки производительных программных систем целиком в этом языке.

Объект и предмет исследования

Объектом данного исследования является система типов CoC и её расширения, а также практическая реализация эффективной среды исполнения CoC.

Предметом исследования являются такие аспекты CoC и её расширений, которые сделают её пригодной для промышленного применения в качестве системы исполнения высокопроизводительных программ, корректных по построению.

³В последнее время стала популярной методика, по которой доказательство математических утверждений ищут с помощью больших языковых моделей, генерирующих программы в одной из таких систем. [22]

⁴В Rosq, однако, есть обходной механизм *извлечения доказательства*, транслирующий скрипт Rosq в программу на OCaml/Haskell/Scheme/...

Цель и задачи исследования

Целью данной работы является поиск системы типов, основанной на CoC, пригодной для написания корректных по построению программ и доступной широкому кругу специалистов в сфере информационных технологий, а также её эффективная реализация.

Для достижения этой цели необходимо выполнить следующие *задачи*:

1. Исследование возможных (как существующих, так и новых) расширений системы CoC;
2. Отбор наиболее необходимых для мейнстримного программирования расширений;
3. Доказательство сохранения важных вычислительных свойств в расширенной системе;
4. Дизайн эргономичного алгоритма вывода типов, поддерживающего существующие и будущие паттерны программирования с зависимыми типами;
5. Реализация эффективной среды исполнения и логического ядра системы;
6. Создание демонстрационных проектов на новом языке, показывающих состоятельность выбранного подхода.

Методология и методы исследования

В силу природы поставленных задач, настоящее исследование производится на стыке математической логики (теория типов, теория моделей), алгебры (символьные вычисления, алгебраическая оптимизация) и теоретической информатики (теория алгоритмов, теория сложности) с акцентом на практической реализации. Для каждой из поставленных задач используются свои методы исследования, в частности:

1. Полученная система типов описывается в декларативной форме, в виде генценовского исчисления секвенций специального вида;
2. Метатеоретические свойства системы доказываются относительно сформулированного исчисления структурной индукцией по дереву естественного вывода в системе;
3. Алгоритм вывода типов также формулируется с помощью исчисления секвенций, но уже в алгоритмической форме, демонстрируется корректность вывода относительно базовой системы;
4. Реализация компилятора и среды исполнения опирается на новейшие практики реализации подобных программных систем;
5. Эффективность среды исполнения проверяется посредством бенчмаркинга и сравнения производительности по сравнению с аналогичными средами на эталонных программах, реализованных на различных языках.

Научная новизна исследования

Данное исследование предлагает новые дизайны для систем типов с зависимыми типами с акцентом на практическую применимость в промышленном программировании, основываясь на новейших результатах в области оснований математики; кроме этого, настоящая работа содержит значимые продвижения в техниках создания эффективных сред исполнения для языков с зависимыми типами, а также в техниках использования зависимых типов в промышленном программировании.

Практическая значимость работы

Существование удобного в использовании языка программирования с эффективной средой исполнения, позволяющего создавать корректные по построению программы, качественно изменит подход к разработке программного обеспечения. На подобные программные решения существует большой спрос, что показывает пример Rust [1]; технология, позволяющая раздвинуть границы возможного ещё дальше, будет востребована не меньше.

Степень достоверности и апробация результатов

В лучших традициях дисциплины *теории типов*, достоверность ключевых теоретических результатов, полученных в ходе исследования, гарантируется формализацией доказательств в среде интерактивных доказательств Rosq. Апробация практических результатов произведена с помощью реализации ряда демонстрационных проектов в полученном языке программирования, показывающих состоятельность и перспективность выбранного подхода.

1. Чистая система типов с рангами функций в качестве сортов

1.1 Мотивация и идея

- Больше всего вопросов при изучении современных средств интерактивных доказательств (Lean, Rocq, Agda...) вызывает устройство *иерархий вселенных*: $\text{Type}_1 : \text{Type}_2 : \dots$
- Вызывают они вопросы и с метатеоретической точки зрения — в доказательствах здравости системы Type_n интерпретируется как n -й недостижимый кардинал.
- Поэтому интересно искать альтернативные варианты устройства иерархии. Далее описан забавный вариант, располагающий типы по “вселенным” соответственно *рангу функций*.
- Бóльшая часть описанных результатов формально верифицирована на Rocq.

1.2 Чистые системы типов

1.2.1 Чистые системы типов: синтаксис

Переменная	x	$\in$	X	
Сорт	s	$\in$	S	
Терм	T, U, t, u	$::=$	x s $\Pi(x : T).U$ $\lambda(x : T).u$ $u\ t$	переменная сорт как тип тип зависимых ф-й анонимная функция применение функции

1.2.2 Чистые системы типов: правила одношаговой редукции

$$\begin{array}{c}
 \frac{T \rightarrow_\beta T'}{\Pi(x : T).U \rightarrow_\beta \Pi(x : T').U} \quad
 \frac{U \rightarrow_\beta U'}{\Pi(x : T).U \rightarrow_\beta \Pi(x : T).U'} \quad
 \frac{T \rightarrow_\beta T'}{\lambda(x : T).t \rightarrow_\beta \lambda(x : T').t} \\
 \\
 \frac{t \rightarrow_\beta t'}{\lambda(x : T).t \rightarrow_\beta \lambda(x : T).t'} \quad
 \frac{t \rightarrow_\beta t'}{t\ u \rightarrow_\beta t'\ u} \quad
 \frac{u \rightarrow_\beta u'}{t\ u \rightarrow_\beta t\ u'} \quad
 (\lambda(x : T).t)\ u \rightarrow_\beta t[x := u]
 \end{array}$$

1.2.3 Чистые системы типов: правила типизации

$$\begin{array}{c}
\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \quad \frac{}{\Gamma \vdash s : s'}(s, s') \in \text{Ax} \quad \frac{\Gamma \vdash T : s_T \quad \Gamma, x : T \vdash U : s_U}{\Gamma \vdash \Pi(x : T).U : s}(s_T, s_U, s) \in \text{Rule} \\
\\
\frac{\Gamma \vdash T : s_T \quad \Gamma, x : T \vdash u : U}{\Gamma \vdash \lambda(x : T).u : \Pi(x : T).U} \quad \frac{\Gamma \vdash u : \Pi(x : T).U \quad \Gamma \vdash t : T}{\Gamma \vdash ut : U[x := t]} \\
\\
\frac{\Gamma \vdash t : T \quad \Gamma \vdash U : s \quad T =_\beta U}{\Gamma \vdash t : U}
\end{array}$$

1.3 «Комбинаторные» свойства ЧСТ

- Отношение многошаговой редукции $(\rightarrow_\beta^*)$ конфлюэнтно, т.е.

$$\begin{array}{ccc}
t & \longrightarrow & t' \\
\downarrow & & \downarrow \\
t'' & \dashrightarrow & t^*
\end{array}$$

- Редукция сохраняет типизацию, т.е.

$$\frac{\Gamma \vdash t : T \quad t \rightarrow_\beta t'}{\Gamma \vdash t' : T}$$

- Здоровы ли ЧСТ? («есть ли полезная модель у ЧСТ?» или «все ли типы населены?» или «любой ли терм нормализуется?»)

1.3.1 О здравости ЧСТ

Какие ЧСТ здоровы?

- Исчисление конструкций:

$$\begin{aligned}
S &= \{*, \square\} \quad \text{Ax} = \{(*, \square)\} \\
\text{Rule} &= \{(*, *, *), (*, \square, \square), (\square, *, *), (\square, \square, \square)\}
\end{aligned}$$

- Первая счётная иерархия *вселенных*:

$$\begin{aligned}
S &= \{\mathcal{U}_\ell \mid \ell \in \mathbb{N}\} \quad \text{Ax} = \{(\mathcal{U}_\ell, \mathcal{U}_{\ell+1}) \mid \ell \in \mathbb{N}\} \\
\text{Rule} &= \{(\mathcal{U}_\ell, \mathcal{U}_{\ell'}, \mathcal{U}_{\max(\ell, \ell')}) \mid \ell, \ell' \in \mathbb{N}\}
\end{aligned}$$

- Счётная иерархия с импредикативным Prop:

$$\begin{aligned}
S &= \{\text{Prop}\} \sqcup \{\mathcal{U}_\ell \mid \ell \in \mathbb{N}\} \\
\text{Ax} &= \{(\text{Prop}, \mathcal{U}_0)\} \sqcup \{(\mathcal{U}_\ell, \mathcal{U}_{\ell+1}) \mid \ell \in \mathbb{N}\} \\
\text{Rule} &= \{(\mathcal{U}_\ell, \text{Prop}, \text{Prop}) \mid \ell \in \mathbb{N}\} \sqcup \\
&\sqcup \{(\mathcal{U}_\ell, \mathcal{U}_{\ell'}, \mathcal{U}_{\max(\ell, \ell')}) \mid \ell, \ell' \in \mathbb{N}\}
\end{aligned}$$

1.3.2 Парадокс Жирара

Теорема 1 (Girard'72). Следующая ЧСТ (известная как *System U*) содержит не нормализующийся терм:

$$S = \{*, \square, \triangle\} \quad \text{Ax} = \{(*, \square), (\square, \triangle)\} \\ \text{Rule} = \{(*, *, *), (\square, *, *), (\triangle, *, *), (\square, \square, \square), (\triangle, \square, \square)\}$$

Более того, этот терм имеет тип $\Pi(T : *).T$.

Другими словами, «здоровая ЧСТ не может поддерживать одновременно **и** импредикативный **Prop**, **и** полиморфные функции».

Из парадокса Жирара выводится противоречивость ещё более простой ЧСТ, известной как Type-in-Type:

$$S = \{*\} \quad \text{Ax} = \{(*, *)\} \quad \text{Rule} = \{(*, *, *)\}$$

1.3.3 Как насчёт рангов?

Рангом функции называется максимальная вложенность Π в позиции аргумента:

$$\text{rk}(\Pi(x : T).U) = \max(\text{rk } T + 1, \text{rk } U)$$

Рассмотрим $S = \{\text{rk}_n \mid n \in \mathbb{N}\}$, и

$$\text{Rule} = \{(\text{rk}_m, \text{rk}_n, \text{rk}_{\max(m+1, n)}) \mid m, n \in \mathbb{N}\}.$$

В качестве Ax можно взять цепочку $\text{rk}_0 : \text{rk}_1 : \text{rk}_2 : \dots$, но получится система, строго менее удобная, чем обычная иерархия вселенных. Попробуем поступить следующим образом:

$$\text{Ax} = \{(\text{rk}_n, \text{rk}_0) \mid n \in \mathbb{N}\}$$

В частности, $\text{rk}_0 : \text{rk}_0$. Но в данной системе парадокс Жирара не применим!

1.4 Файл AbstractRewriting.v

В данном файле развиваются основные механизмы абстрактного переписывания над произвольным отношением на множестве.

Файл импортирует библиотеку надмножеств (*setoid library*) из стандартной библиотеки.

Определение 1 (Рефлексивно-транзитивная обёртка). Для множества A и отношения R на A рефлексивно-транзитивная обёртка RTC отношения R есть наименьшее отношение на A , генерируемое двумя конструкторами: конструктор рефлексивности, дающий $x \text{ RTC } x$ для каждого $x \in A$; и конструктор шага, который, по заданным $x R y$ и $y \text{ RTC } z$, даёт $x \text{ RTC } z$.

Определение 2 (Симметрическая обёртка). Симметрическая обёртка Sym отношения R есть наименьшее отношение на A , генерируемое: прямым конструктором, который, по заданному $x R y$, даёт $x \text{ Sym } y$; и обратным конструктором, который, по заданному $y R x$, даёт $x \text{ Sym } y$.

Определение 3 (Обратное отношение). Обратное отношение R^{-1} определяется так: $x R^{-1} y$ тогда и только тогда, когда $y R x$.

Определение 4 (Эквивалентная обёртка). Эквивалентная обёртка отношения R есть рефлексивно-транзитивная обёртка симметрической обёртки: $\text{EquivClosure } R \triangleq \text{RTC} (\text{Sym } R)$.

Определение 5 (Прообраз отношения). Для функции $f : A \rightarrow B$ и отношения R на B прообраз отношения R по f (обозначение $R @ f$) есть отношение на A , определённое тем, что $x (R @ f) y$ тогда и только тогда, когда $f(x) R f(y)$.

Определение 6 (Свойство Церча–Россера). Отношение R на A обладает свойством Церча–Россера, если для всех $t, t_1, t_2 \in A$, из $t R t_1$ и $t R t_2$ существует и такой, что $t_1 R$ и $t_2 R$.

Экземпляр 1. Рефлексивно-транзитивная обёртка любого отношения рефлексивна.

Экземпляр 2. Рефлексивно-транзитивная обёртка любого отношения транзитивна.

Лемма 1. Если $x R y$, то $x \text{ RTC } y$.

Экземпляр 3. Если R симметрично, то его рефлексивно-транзитивная обёртка также симметрична.

Экземпляр 4. Симметрическая обёртка любого отношения симметрична.

Экземпляр 5. Эквивалентная обёртка любого отношения есть отношение эквивалентности (т. е. рефлексивна, симметрична и транзитивна).

Лемма 2. Если $x \text{ RTC } (R^{-1}) y$, то $y \text{ RTC } (R) x$.

Лемма 3 (Отображение через рефлексивно-транзитивную обёртку). Для функции $f : A \rightarrow B$ и отношения R на B : если $x \text{ RTC } (R @ f) y$, то $f(x) \text{ RTC } (R) f(y)$.

Лемма 4 (Монотонность рефлексивно-транзитивной обёртки). Если $Q \subseteq R$ (т. е. $Q x y \Rightarrow R x y$ для всех x, y), то $\text{RTC } Q \subseteq \text{RTC } R$.

Лемма 5 (Отображение через симметрическую обёртку). Для функции $f : A \rightarrow B$ и отношения R на B : если $x \text{ Sym } (R @ f) y$, то $f(x) \text{ Sym } (R) f(y)$.

Лемма 6 (Монотонность симметрической обёртки). Если $Q \subseteq R$, то $\text{Sym } Q \subseteq \text{Sym } R$.

Лемма 7. Если $x R y$, то $x \text{ EquivClosure } (R) y$.

Лемма 8 (Отображение через эквивалентную обёртку). Для функции $f : A \rightarrow B$ и отношения R на B : если $x \text{ EquivClosure } (R @ f) y$, то $f(x) \text{ EquivClosure } (R) f(y)$.

Лемма 9 (Монотонность эквивалентной обёртки). Если $Q \subseteq R$, то $\text{EquivClosure } (Q) \subseteq \text{EquivClosure } (R)$.

Лемма 10 (Экстенциональность свойства Церча–Россера). Если два отношения Q и R на A равны (т. е. $Q(a, b) \Leftrightarrow R(a, b)$ для всех a, b), то Q обладает свойством Церча–Россера тогда и только тогда, когда R обладает свойством Церча–Россера.

Теорема 2 (Свойство Церча–Россера для рефлексивно-транзитивной обёртки). Если отношение R обладает свойством Церча–Россера, то его рефлексивно-транзитивная обёртка $\text{RTC } R$ также обладает свойством Церча–Россера.

1.5 Файл Syntax.v

В данном файле определяется синтаксис зависимой теории типов с одним сортом типов, произведениями, лямбдами, приложениями и переменными, а также базовые операции манипуляции термами.

Файл импортирует модуль проверенного равенства из стандартной библиотеки.

Определение 7 (Ограниченные натуральные числа). Тип $\text{Fin}(n)$ состоит из натуральных чисел, строго меньших n , с конструкторами fzero (первый элемент) и fsucc (преемник элемента).

Определение 8 (Термы). Для множества уровней L и натурального числа n (количество переменных в контексте) терм $\text{Term}(L, n)$ есть одно из: вселенная $\mathcal{U}(\ell)$ для уровня $\ell \in L$; произведение $\Pi T U$, где T — терм в контексте длины n , а U — терм в контексте длины $n + 1$; переменная $\text{var}(i)$ для индекса де Брюйна $i \in \text{Fin}(n)$; лямбда $\lambda T t$, где T — терм в контексте длины n , а t — терм в контексте длины $n + 1$; или применение $f \$$ и двух термов в контексте длины n .

Определение 9 (Матчинг по Fin). Вспомогательная функция, которая, по заданному значению z для базового случая и функции s для индуктивного случая, производит матчинг по индексу де Брюйна.

Определение 10 (Ослабление функции переименования). Для функции переименования $f : \text{Fin}(m) \rightarrow \text{Fin}(n)$ ослабление $\text{weak}(f) : \text{Fin}(S m) \rightarrow \text{Fin}(S n)$ отображает новый индекс fzero на fzero и сдвигает остальные.

Определение 11 (Переименование). Для функции переименования $f : \text{Fin}(m) \rightarrow \text{Fin}(n)$ операция переименования преобразует терм в контексте длины m в терм в контексте длины n : вселенные остаются без изменений; в произведении область переименовывается по f , а тело — по $\text{weak}(f)$; индексы переменных отображаются по f ; в лямбде область переименовывается по f , а тело — по $\text{weak}(f)$; в применении обе части переименовываются по f .

Определение 12 (Сдвиг). Сдвиг терма t в контексте длины n даёт терм в контексте длины $n + 1$ путём увеличения каждого индекса переменной (т. е. переименование по функции, отображающей каждый индекс на его преемник).

Определение 13 (Транспонирование). Для функции $f : \text{Fin}(m) \rightarrow \text{Term}(L, n)$ транспонирование по индексу i даёт терм в контексте длины $n + 1$: при $i = \text{fzero}$ это переменная $\text{var}(\text{fzero})$; при $i = \text{fsucc}(j)$ это $\text{shift}(f(j))$.

Определение 14 (Подстановка). Для функции $f : \text{Fin}(m) \rightarrow \text{Term}(L, n)$ и терма t в контексте длины m операция подстановки заменяет переменные в t соответствующими термами из f : вселенные остаются без изменений; в произведении область подставляется по f , а тело — по транспонированию f ; переменная i заменяется на $f(i)$; в лямбде область подставляется по f , а тело — по транспонированию f ; в применении обе части подставляются по f .

Определение 15 (Подстановка в свежую переменную). Подстановка терма u в свежую переменную терма t (обозначение $t \triangleleft u$) есть операция замены переменной с индексом fzero на u при сохранении остальных переменных со сдвигом.

Определение 16 (Толкание). Для терма t в контексте длины n функция толкания производит, для каждого индекса i в $\text{Fin}(m + n)$, соответствующий терм в контексте длины $m + n$ с надлежащим сдвигом.

Определение 17 (Индексная подстановка). Подстановка терма u в переменную с заданным индексом в терме t (обозначение $t \triangleleft^i u$) выполняется путём толкания u к нужному индексу i последующего применения подстановки.

Лемма 11 (Экстенциональность переименования). Если две функции переименования f и g совпадают на всех индексах, то они дают одинаковый результат при применении к любому терму.

Лемма 12 (Экстенциональность транспонирования). Если две функции f и g совпадают на всех индексах, то их транспонирования совпадают на всех индексах.

Лемма 13 (Экстенциональность подстановки). Если две функции подстановки f и g совпадают на всех индексах, то они дают одинаковый результат при применении к любому терму.

Лемма 14 (Композиция переименований). Переименование по f после переименования по g равносильно переименованию по композиции $f \circ g$.

Лемма 15 (Переименование коммутрует с подстановкой). Переименование подставленного терма по f равносильно подстановке по переименованной функции с последующим применением результата.

Лемма 16 (Подстановка коммутрует с переименованием). Подстановка в переименованный терм равносильна подстановке по композиции функций.

Лемма 17 (Подстановка с тождественным отображением). Замена переменных на самих себя есть тождественная операция на термах.

Лемма 18 (Композиция подстановок). Подстановка одной подстановки в другую равносильна подстановке по композиции функций.

Лемма 19 (Переименование распределяется над подстановкой). Переименование подстановки $t \triangleleft u$ по f равносильно подстановке $\text{weak}(f)(t) \triangleleft f(u)$.

Лемма 20 (Подстановка распределяется над подстановкой). Подстановка в подстановку $t \triangleleft u$ по f равносильна подстановке $\text{transpose}(f)(t) \triangleleft f(u)$.

Лемма 21 (Сдвиг, затем подстановка, — тождественно). Сдвиг терма u и последующая подстановка переменной в свежий индекс восстанавливают исходный терм.

Лемма 22 (Сдвиг коммутрует с индексной подстановкой). Сдвиг, за которым следует индексная подстановка, равносильна индексной подстановке, за которой следует сдвиг.

1.6 Файл `Congruence.v`

В данном файле определяется понятие конгруэнтного отношения на синтаксисе термов и доказано, что оно сохраняется при симметрической и рефлексивно-транзитивной обёртках.

Файл импортирует библиотеку надмножеств из стандартной библиотеки, а также модули синтаксиса и абстрактного переписывания.

Класс 1 (Конгруэнтность). Конгруэнтность для семейства отношений R на термах (индексированного длиной контекста) — это набор из шести условий совместимости: отношение совместимо с производными по левому аргументу (если $T R T'$, то $\Pi T U R \Pi T' U$) и по правому (если $U R U'$, то $\Pi T U R \Pi T U'$); совместимо с лямбдами по левому аргументу (если $T R T'$, то $\lambda T t R \lambda T' t$) и по правому (если $t R t'$, то $\lambda T t R \lambda T t'$); совместимо с приложениями по левому аргументу (если $t R t'$, то $t \$ u R t' \$ u$) и по правому (если $u R u'$, то $t \$ u R t \$ u'$).

Экземпляр 6 (Симметрическая обёртка сохраняет конгруэнтность). Если R — конгруэнтность, то семейство симметрических обёрток $\text{Sym}(R)$ также является конгруэнтностью.

Экземпляр 7 (Рефлексивно-транзитивная обёртка сохраняет конгруэнтность). Если R — конгруэнтность, то семейство рефлексивно-транзитивных обёрток $\text{RTC}(R)$ также является конгруэнтностью.

Лемма 23 (Параллельная конгруэнтность для произведений). Если R — транзитивная конгруэнтность, и $T R T', U R U'$, то $\Pi T U R \Pi T' U'$.

Лемма 24 (Параллельная конгруэнтность для лямбд). Если R — транзитивная конгруэнтность, и $T R T', t R t'$, то $\lambda T t R \lambda T' t'$.

Лемма 25 (Параллельная конгруэнтность для приложений). Если R — транзитивная конгруэнтность, и $t R t', u R u'$, то $t \$ u R t' \$ u'$.

1.7 Файл Reduction.v

В данном файле определяется одношаговое β -преобразование и доказаны его базовые свойства.

Файл импортирует модули отношений и проверенного равенства из стандартной библиотеки, а также модули синтаксиса, конгруэнтности и абстрактного переписывания.

Определение 18 (Одношаговое β -преобразование). Отношение $\xrightarrow{\beta}$ на термах в заданном контексте определяется семью конструкторами: преобразование в области произведения ($\Pi T U \rightarrow \Pi T' U$, если $T \rightarrow T'$); преобразование в теле произведения ($\Pi T U \rightarrow \Pi T U'$, если $U \rightarrow U'$); преобразование в области лямбды ($\lambda T t \rightarrow \lambda T' t$, если $T \rightarrow T'$); преобразование в теле лямбды ($\lambda T t \rightarrow \lambda T t'$, если $t \rightarrow t'$); преобразование в функции приложения ($f \$ t \rightarrow f' \$ t$, если $f \rightarrow f'$); преобразование в аргументе приложения ($f \$ t \rightarrow f \$ t'$, если $t \rightarrow t'$); и β -правило ($\lambda T f \$ t \rightarrow f \triangleleft t$).

Экземпляр 8. Одношаговое β -преобразование является конгруэнтностью.

Определение 19 (Многошаговое β -преобразование). Отношение $\xrightarrow{*}_{\beta}$ есть рефлексивно-транзитивная обёртка отношения $\xrightarrow{\beta}$.

Лемма 26 (Переименование коммутрует с шагом). Если $t \xrightarrow{\beta} t'$, то $\text{rename}(f)(t) \xrightarrow{\beta} \text{rename}(f)(t')$.

Лемма 27 (Подстановка коммутует с шагом). Если $t \xrightarrow{\beta} t'$, то $\text{replace}(f)(t) \xrightarrow{\beta} \text{replace}(f)(t')$.

Лемма 28 (Лямбда-инверсия для переименования). Если лямбда-терм равен переименованию некоторого терма, то исходный терм сам является лямбдой, и переименование раскладывается соответственно.

Лемма 29 (Инверсия шага для переименования). Если $\text{rename}(f)(t) \xrightarrow{\beta} u$, то существует v такой, что $u = \text{rename}(f)(v)$ и $t \xrightarrow{\beta} v$.

Лемма 30 (Переименование коммутует с многошаговым преобразованием). Если $t \xrightarrow{*}_{\beta} t'$, то $\text{rename}(f)(t) \xrightarrow{*}_{\beta} \text{rename}(f)(t')$.

Лемма 31 (Транспонирование коммутует с многошаговым преобразованием). Если каждый компонент функции подстановки преобразуется в соответствующий компонент другой, то их транспонирования также преобразуются друг в друга.

Лемма 32 (Подстановка коммутует с многошаговым преобразованием). *Если каждый компонент функции подстановки преобразуется в соответствующий компонент другой, то подставленные термы также преобразуются друг в друга.*

Лемма 33 (Представление произведения). *Если $\Pi T U \xrightarrow{*}_\beta t$, то t является произведением.*

Лемма 34 (Инверсия произведения для многошагового преобразования). *Если $\Pi T U \xrightarrow{*}_\beta \Pi T' U'$, то $T \xrightarrow{*}_\beta T'$ и $U \xrightarrow{*}_\beta U'$.*

1.8 Файл Confluence.v

В данном файле определяется отношение параллельного β -преобразования и доказывается свойство связности (церх–россер) как для параллельного, так и для стандартного β -преобразования.

Файл импортирует стандартную библиотеку, модули абстрактного переписывания, синтаксиса, редукции и конгруэнтности.

Определение 20 (Параллельное β -преобразование). *Отношение параллельной редукции $\twoheadrightarrow_\beta$ определяется: рефлексивным шагом ($t \twoheadrightarrow_\beta t$); параллельным преобразованием в области и теле произведения; параллельным преобразованием в области и теле лямбды; параллельным преобразованием в функции и аргументе приложения; и β -шагом, в котором функция и аргумент могут содержать внутренние параллельные преобразования ($\lambda T f \$ t \twoheadrightarrow_\beta f' \triangleleft t'$, если $f \twoheadrightarrow_\beta f'$ и $t \twoheadrightarrow_\beta t'$).*

Лемма 35 (Один шаг — параллельный шаг). *Если $t \xrightarrow{\beta} t'$, то $t \twoheadrightarrow_\beta t'$.*

Лемма 36 (Параллельный шаг — многошаговое преобразование). *Если $t \twoheadrightarrow_\beta t'$, то $t \xrightarrow{*}_\beta t'$.*

Лемма 37 (Переименование коммутует с параллельным шагом). *Если $t \twoheadrightarrow_\beta t'$, то $\text{rename}(f)(t) \twoheadrightarrow_\beta \text{rename}(f)(t')$.*

Лемма 38 (Транспонирование коммутует с параллельным шагом). *Если каждый компонент функции подстановки делает параллельный шаг в соответствующий компонент другой, то их транспонирования также делают параллельный шаг.*

Лемма 39 (Подстановка коммутует с параллельным шагом). *Если каждый компонент функции подстановки делает параллельный шаг в соответствующий компонент другой, и $t \twoheadrightarrow_\beta t'$, то подставленные термы также делают параллельный шаг друг в друга.*

Лемма 40 (Подстановка распределяется над параллельным шагом). *Если $t \twoheadrightarrow_\beta t'$, то $f \triangleleft t \twoheadrightarrow_\beta f \triangleleft t'$.*

Лемма 41 (Параллельные шаги в функции и аргументе). *Если $f \twoheadrightarrow_\beta f'$ и $t \twoheadrightarrow_\beta t'$, то $f \triangleleft t \twoheadrightarrow_\beta f' \triangleleft t'$.*

Лемма 42 (Лямбда-инверсия для параллельного шага). *Если лямбда-терм делает параллельный шаг в некоторый терм t' , то t' является лямбдой, и параллельный шаг раскладывается на параллельные шаги в области и теле.*

Лемма 43 (Двойниковое свойство для параллельной редукции). *Отношение параллельного β -преобразования обладает свойством Церча–Россера.*

Теорема 3 (Связность β -преобразования). *Отношение стандартного β -преобразования обладает свойством Церча–Россера (т. е. β -преобразование связно).*

1.9 Файл DefEq.v

В данном файле определяется тождественное равенство (эквивалентная обёртка одного β -шага) и доказаны его базовые свойства.

Файл импортирует стандартную библиотеку, модули абстрактного переписывания, синтаксиса, редукции и связности.

Определение 21 (Тождественное равенство). *Два терма t и t' равны тождественно (обозначение $t =_\beta t'$), если они связаны эквивалентной обёрткой отношения одношагового β -преобразования.*

Теорема 4 (Характеризация тождественного равенства через Церча–Россера). *Два терма равны тождественно тогда и только тогда, когда существует общий результат редукции: т. е. $t =_\beta t'$ тогда и только тогда, когда существует терм u такой, что $t \xrightarrow{*}_\beta u$ и $t' \xrightarrow{*}_\beta u$.*

Лемма 44 (Инверсия произведения для тождественного равенства). *Если $\Pi T U =_\beta \Pi T' U'$, то $T =_\beta T'$ и $U =_\beta U'$.*

Лемма 45 (Подстановка сохраняет тождественное равенство). *Если $t =_\beta t'$, то $f \triangleleft t =_\beta f \triangleleft t'$.*

Лемма 46 (Переименование сохраняет тождественное равенство). *Если $t =_\beta t'$, то $\text{rename}(f)(t) =_\beta \text{rename}(f)(t')$.*

Лемма 47 (Подстановка сохраняет тождественное равенство). *Если $t =_\beta t'$, то $\text{replace}(f)(t) =_\beta \text{replace}(f)(t')$.*

1.10 Файл Context.v

В данном файле определяется синтаксис контекстов, отношение β -преобразования на контекстах и вспомогательные операции манипуляции контекстами.

Файл импортирует модули проверенного равенства и отношений из стандартной библиотеки, а также модули синтаксиса и редукции.

Определение 22 (Контексты). *Контекст длины n — это либо пустой контекст ε (длины 0), либо расширение $\Gamma \& T$ контекста Γ длины n термом T в контексте длины n , дающее контекст длины $n + 1$.*

Определение 23 (β -преобразование контекстов). *Отношение $\xrightarrow{\beta}$ на контекстах определяется: β -преобразованием в начале контекста (если $T \xrightarrow{\beta} U$, то $\Gamma \& T \xrightarrow{\beta} \Gamma \& U$); и β -преобразованием глубже в контексте (если $\Gamma \xrightarrow{\beta} \Delta$, то $\Gamma \& T \xrightarrow{\beta} \Delta \& T$).*

Определение 24 (Предшественник контекста). *Для непустого контекста $\Gamma = \Gamma_0 \& T$ предшественник есть Γ_0 .*

Определение 25 (Вершина контекста). *Для непустого контекста $\Gamma = \Gamma_0 \& T$ вершина (или головной элемент) есть T .*

Определение 26 (Индексирование контекста). *Для контекста Γ длины n и индекса $i \in \text{Fin}(n)$ обозначение $\Gamma !! i$ извлекает i -й тип из контекста (считая с конца), с надлежащим сдвигом.*

Определение 27 (Переименователь ослабления). *Функция $\text{wr} : \text{Fin}(m+n) \rightarrow \text{Fin}(S(m+n))$, сдвигающая индексы с учётом нового типа, вставленного в начало контекста.*

Определение 28 (Вставка в контекст). Вставка типа T в начало контекста Γ длины $m+n$ даёт контекст длины $S(m+n)$ с надлежащим переименованием существующих типов.

Определение 29 (Сжатие контекста). Удаление первых m типов из контекста длины $m+n$, оставляя контекст длины n .

Определение 30 (Индексная подстановка в контекст). Подстановка терма t в переменную с заданной позицией в контексте, дающая новый контекст, в котором этот тип заменён на t с надлежащим индексом.

Определение 31 (Сжатие до одного элемента). Извлечение одного типа из контекста длины $S(m+n)$ путём снятия первых m типов.

Лемма 48. Применение переименователя ослабления к преемнику индекса равно преемнику применения переименователя ослабления к индексу.

Лемма 49. Ослабление переименователя ослабления равно самому переименователю ослабления на преемниках индексов.

Лемма 50. Переименование по ослаблению переименователя ослабления равносильно переименованию по переименователю ослабления.

Лемма 51 (Преимник индекса). Индексирование контекста, расширенного новым типом, по преемнику индекса даёт сдвиг индекса в исходном контексте.

Лемма 52 (Вставка и индексирование). Индексирование вставленного контекста по ослабленному индексу даёт переименование исходного индекса.

1.11 Файл Levels.v

В данном файле определяется сигнатура структуры уровней вселенных и дополнительные условия функциональности и полноты.

Класс 2 (Сигнатура уровней). Сигнатура уровней на типе L состоит из бинарного отношения `axiom` (представляющего, что один уровень является аксиомой на другом уровне) и трёхместного отношения `rule` (представляющего формирование типа произведения из уровней области и области значений).

Класс 3 (Функциональная сигнатура уровней). Сигнатура уровней функциональна, если: отношение аксиомы детерминировано (если `axiom( $\ell, \ell_1$ )` и `axiom( $\ell, \ell_2$ )`, то $\ell_1 = \ell_2$); и отношение правила детерминировано по результату (если `rule( $\ell_t, \ell_u, \ell_1$ )` и `rule( $\ell_t, \ell_u, \ell_2$ )`, то $\ell_1 = \ell_2$).

Класс 4 (Полная сигнатура уровней). Сигнатура уровней полна, если: каждый уровень имеет аксиому (для каждого ℓ существует ℓ' такой, что `axiom( $\ell, \ell'$ )`); и каждая пара уровней определяет уровень произведения (для каждого ℓ_t, ℓ_u существует ℓ такой, что `rule( $\ell_t, \ell_u, \ell$ )`).

1.12 Файл Typing.v

В данном файле определяется суждение типирования и доказаны ключевые метатеоретические леммы ослабления и подстановки.

Файл импортирует стандартную библиотеку, а также модули контекстов, тождественного равенства, уровней и синтаксиса.

Определение 32 (Суждение типирования). Суждение $\Gamma \vdash t \Leftarrow T$ (читается: «в контексте Γ терм t имеет тип T ») определяется индуктивно следующими правилами:

- **Вселенная:** Если $\text{axiom}(\ell, \ell')$, то $\Gamma \vdash \mathcal{U}(\ell) \Leftarrow \mathcal{U}(\ell')$.
- **Произведение:** Если $\Gamma \vdash T \Leftarrow \mathcal{U}(\ell_t)$, $\Gamma \& T \vdash U \Leftarrow \mathcal{U}(\ell_u)$ и $\text{rule}(\ell_t, \ell_u, \ell)$, то $\Gamma \vdash \Pi T U \Leftarrow \mathcal{U}(\ell)$.
- **Переменная:** $\Gamma \vdash \text{var}(i) \Leftarrow \Gamma !! i$.
- **Лямбда:** Если $\Gamma \vdash U \Leftarrow \mathcal{U}(\ell)$ и $\Gamma \& U \vdash t \Leftarrow T$, то $\Gamma \vdash \lambda U t \Leftarrow \Pi U T$.
- **Применение:** Если $\Gamma \vdash t \Leftarrow \Pi U T$ и $\Gamma \vdash u \Leftarrow U$, то $\Gamma \vdash t \$ u \Leftarrow T \triangleleft u$.
- **Конверсия:** Если $\Gamma \vdash t \Leftarrow T$, $\Gamma \vdash T' \Leftarrow \mathcal{U}(\ell)$ и $T =_\beta T'$, то $\Gamma \vdash t \Leftarrow T'$.

Лемма 53 (Сильное ослабление). Если $\Gamma \vdash t \Leftarrow T$ и мы вставляем новый тип U в начало контекста, то переименованный терм и тип остаются корректно типированными: $\text{insert}(\Gamma, U) \vdash \text{rename}(\text{wr})(t) \Leftarrow \text{rename}(\text{wr})(T)$.

Лемма 54 (Ослабление). Если $\Gamma \vdash t \Leftarrow T$, то для любого типа U $\Gamma \& U \vdash \text{shift}(t) \Leftarrow \text{shift}(T)$.

Лемма 55 (Сильная подстановка). Если $\Gamma \vdash t \Leftarrow T$ и $\text{shrink}(\Gamma) \vdash u \Leftarrow \text{squeeze}(\Gamma)$, то подстановка u в контекст и терм/тип сохраняет суждение типирования.

Лемма 56 (Подстановка). Если $\Gamma \& U \vdash t \Leftarrow T$ и $\Gamma \vdash u \Leftarrow U$, то $\Gamma \vdash t \triangleleft u \Leftarrow T \triangleleft u$.

1.13 Файл Inversions.v

В данном файле доказываются леммы инверсии, восстанавливающие структуру вывода типирования из его заключения.

Файл импортирует стандартную библиотеку, а также модули абстрактного переписывания, тождественного равенства, уровней, типирования, контекстов, синтаксиса, конгруэнтности и редукции.

Лемма 57 (Инверсия вселенной). Если $\Gamma \vdash \mathcal{U}(\ell) \Leftarrow T$, то существует уровень ℓ' такой, что $\text{axiom}(\ell, \ell')$ и $T =_\beta \mathcal{U}(\ell')$.

Лемма 58 (Инверсия произведения). Если $\Gamma \vdash \Pi T U \Leftarrow V$, то существуют уровни ℓ_t, ℓ_u, ℓ такие, что $\text{rule}(\ell_t, \ell_u, \ell)$, $\Gamma \vdash T \Leftarrow \mathcal{U}(\ell_t)$, $\Gamma \& T \vdash U \Leftarrow \mathcal{U}(\ell_u)$ и $V =_\beta \mathcal{U}(\ell)$.

Лемма 59 (Инверсия переменной). Если $\Gamma \vdash \text{var}(i) \Leftarrow T$, то $T =_\beta \Gamma !! i$.

Лемма 60 (Инверсия лямбды). Если $\Gamma \vdash \lambda T t \Leftarrow V$, то существуют уровень ℓ и терм U такие, что $\Gamma \vdash T \Leftarrow \mathcal{U}(\ell)$, $\Gamma \& T \vdash t \Leftarrow U$ и $V =_\beta \Pi T U$.

Лемма 61 (Инверсия применения). Если $\Gamma \vdash t \$ u \Leftarrow V$, то существуют термы U и T такие, что $\Gamma \vdash t \Leftarrow \Pi U T$, $\Gamma \vdash u \Leftarrow U$ и $V =_\beta T \triangleleft u$.

Теорема 5 (Единственность типирования / Единственность типа). Если сигнатура уровней функциональна, и $\Gamma \vdash t \Leftarrow T$ и $\Gamma \vdash t \Leftarrow T'$, то $T =_\beta T'$. Иными словами, терм имеет не более одного типа с точностью до тождественного равенства.

1.14 Файл `WF.v`

В данном файле определяется хорошо определённость контекстов.

Файл импортирует модули уровней, контекстов, типирования и синтаксиса.

Определение 33 (Хорошо определённые контексты). *Контекст Γ хорошо определён (обозначение $\text{wf}(\Gamma)$), если: пустой контекст хорошо определён; и если Γ хорошо определён и $\Gamma \vdash T \Leftarrow \mathcal{U}(\ell)$ для некоторого уровня ℓ , то расширение $\Gamma \& T$ хорошо определено.*

1.15 Файл `Metatheory.v`

В данном файле доказывается свойство сохранения типов (*subject reduction*).

Файл импортирует стандартную библиотеку, а также модули типирования, контекстов, хорошо определённости, уровней, синтаксиса, тождественного равенства, редукции, абстрактного переписывания, инверсий и конгруэнтности.

Лемма 62 (Сильная редукция контекста). *Если $\Gamma \vdash t \Leftarrow T$, Γ хорошо определён, и $\Gamma \xrightarrow{\beta} \Delta$ (шаг β -преобразования в контексте), то $\Delta \vdash t \Leftarrow T$. Иными словами, редукция типа в контексте сохраняет типирование терма.*

Лемма 63 (Редукция контекста). *Если $\Gamma \& U \vdash t \Leftarrow T$, Γ хорошо определён, $\Gamma \vdash U \Leftarrow \mathcal{U}(\ell)$ и $U \xrightarrow{\beta} U'$, то $\Gamma \& U' \vdash t \Leftarrow T$. Иными словами, редукция типа в начале контекста сохраняет типирование терма в расширенном контексте.*

Лемма 64 (Корректность типа). *Если сигнатура уровней полна, и $\Gamma \vdash t \Leftarrow T$ при том, что Γ хорошо определён, то существует уровень ℓ такой, что $\Gamma \vdash T \Leftarrow \mathcal{U}(\ell)$. Иными словами, тип в суждении типирования сам является хорошо типированным как вселенная.*

Теорема 6 (Сохранение типов). *Если сигнатура уровней полна, и $\Gamma \vdash t \Leftarrow T$ при том, что Γ хорошо определён, и $t \xrightarrow{\beta} t'$, то $\Gamma \vdash t' \Leftarrow T$. Иными словами, β -преобразование сохраняет типирование: если терм имеет тип, то его β -редукт также имеет тот же тип.*

1.16 Файл `Atomic.v`

В данном файле определяется понятие атомарного терма и доказывается сохранение атомарности при переименовании и β -преобразовании.

Файл импортирует стандартную библиотеку, а также модули синтаксиса и редукции.

Определение 34 (Атомарные термы). *Терм атомарен, если он есть: вселенная; произведение; переменная; или применение атомарной функции к атомарному аргументу.*

Лемма 65 (Атомарность сохраняется при переименовании). *Если терм атомарен, то любое его переименование также атомарно.*

Лемма 66 (Атомарность сохраняется при β -преобразовании). *Если $t \xrightarrow{\beta} t'$ и t атомарен, то t' также атомарен.*

1.17 Файл StronglyNormalizing.v

В данном файле определяется сильная нормализуемость и доказываются ряд вспомогательных лемм.

Файл импортирует стандартную библиотеку, а также модули синтаксиса, редукции и атомарности.

Определение 35 (Доступность / Обоснованность). Для отношения R на множестве A элемент x доступен (относительно R), если каждый y такой, что $x R y$, сам доступен.

Определение 36 (Сильная нормализуемость). Терм t сильно нормализуем (обозначение $SN(t)$), если он доступен относительно отношения одношагового β -преобразования.

Лемма 67 (Сильная нормализуемость по голове). Если $t \ \$$ и сильно нормализуем, то t сильно нормализуем.

Лемма 68 (Сильная нормализуемость атомарного применения). Если t атомарен и сильно нормализуем, и u сильно нормализуем, то $t \ \$$ и сильно нормализуем.

Лемма 69 (Переименование сохраняет сильную нормализуемость). Если терм сильно нормализуем, то любое его переименование также сильно нормализуемо.

Определение 37 (Слабое головное расширение). Отношение WHE (слабое головное расширение) на термах определяется: если t слабо головно расширяется в t' , то $t \ \$$ и слабо головно расширяется в $t' \ \$$; и если U и u сильно нормализуемы, то $\lambda U t \ \$$ и слабо головно расширяется в $t \triangleleft u$.

Определение 38 (Аджасть). Функция, которая для заданного m отображает индекс в $Fin(n)$ в индекс в $Fin(m + n)$ путём сдвига всех индексов на m .

Лемма 70 (Аджасть и сильная нормализуемость). Если терм, переименованный функцией $adjust$, сильно нормализуем, то исходный терм сильно нормализуем.

Лемма 71 (Переименование коммутрует со слабым головным расширением). Если t слабо головно расширяется в t' , то любое переименование t слабо головно расширяется в переименование t' .

Лемма 72 (Слабое головное расширение и сильная нормализуемость). Если t слабо головно расширяется в t' и $t' \ \$$ и сильно нормализуем, то $t \ \$$ и сильно нормализуем. Эта лемма объявлена аксиомой в ожидании исправления на основе доказательства Альтенкирха.

1.18 Файл LambdaSet.v

В данном файле определяется рамка Λ -множеств, которая обеспечивает семантическую модель для теории типов с помощью насыщенных множеств реализаторов.

Файл импортирует стандартную библиотеку, а также модули надмножеств, синтаксиса, сильной нормализуемости, атомарности и редукции.

Класс 5 (Λ -множество). Λ -множество X_0 над языком L — это множество моделей (или реализаторов): для каждой длины контекста n и терма t — предложение $t \models x$ для каждого элемента $x \in X_0$.

Определение 39 (Насыщенное множество). Семейство предикатов $\mathcal{C}$ на термах насыщено, если: (1) каждый терм из $\mathcal{C}$ сильно нормализуем; (2) каждый атомарный сильно нормализуемый терм принадлежит $\mathcal{C}$; и (3) $\mathcal{C}$ замкнуто относительно слабого головного расширения (если t слабо головно расширяется в t' и $t' \in \mathcal{C}$, то $t \in \mathcal{C}$).

Класс 6 (Насыщенное Λ -множество). Насыщенное Λ -множество — это Λ -множество X_0 с: свойством сильной нормализуемости (если $t \models x$, то t сильно нормализуем); выделенным центром; свойством централизации реализации (каждый атомарный сильно нормализуемый терм реализует центр); и замкнутостью относительно расширения (если t слабо головно расширяется в t' и $t' \models x$, то $t \models x$).

Замечание 1 (Реализаторы образуют насыщенное множество). В насыщенном Λ -множестве множество всех термов, реализующих некоторый элемент, насыщено.

Определение 40 (Λ -морфизм). Λ -морфизм между двумя Λ -множествами X_0 и Y_0 — это функция $p : X_0 \rightarrow Y_0$, совместимая с отношением реализации: если $t \models x$, то $t \models p(x)$.

Определение 41 (Λ -изоморфизм). Λ -изоморфизм между двумя Λ -множествами X_0 и Y_0 состоит из прямой карты, обратной карты, доказательств того, что они взаимно обратны, и доказательства того, что прямая карта сохраняет отношение реализации (т. е. $t \models x$ тогда и только тогда, когда $t \models \text{forward}(x)$).

Определение 42 (Обратный изоморфизм). Для заданного Λ -изоморфизма его обратный также является Λ -изоморфизмом.

Класс 7 ($\mathcal{E}$ -множество). $\mathcal{E}$ -множество $\mathfrak{A}$ над носителем A и языком L с окружением $\mathcal{E}$ есть структура, состоящая из: для каждого $\alpha \in \mathfrak{A}$ — отношения $\Lambda_{\text{set}}(\alpha) : A \rightarrow \text{Prop}$ (Λ -множество, индексированное по α); для каждого элемента окружения $i \in \mathcal{E}$ — отношения эквивалентности $\mathfrak{A} \sim_i$ на множестве индексов $\mathfrak{A}$; для каждого i — отношения эквивалентности на носителе A ; и свойства носителя, гласящего, что если два индекса связаны отношением $\sim_i$ и соответствующие реализаторы совпадают на всех атомарных сильно нормализуемых термах, то соответствующие носители связаны отношением эквивалентности носителя.

Определение 43 (Принадлежность индекса). Обозначение $\alpha \sqsubset X$ означает, что α является элементом Λ -множества X .

Определение 44 (Связность индексов). Обозначение $\alpha \langle i \rangle \alpha'$ означает, что α и α' связаны i -м отношением эквивалентности.

Определение 45 (Эквивалентность носителей). Обозначение $x \langle i \rangle y$ означает, что носители x и y связаны i -м отношением эквивалентности носителей.

Определение 46 (Эквивалентность носителей с контекстом). Обозначение $x \langle i \mid X \rangle y$ явно фиксирует контекст X в эквивалентности носителей.

Определение 47 (Внутреннее произведение Π'). Для $\mathcal{E}$ -множества $\mathfrak{A}$ и семейства $Y : \mathfrak{A} \rightarrow \mathfrak{B}$ $\mathcal{E}$ -множеств внутреннее произведение $\Pi' X Y$ есть тип семейств f , которые для каждого $\alpha \in \mathfrak{A}$ назначают элемент из $Y(\alpha)$, удовлетворяющих условию согласованности: связанные индексы дают связанные элементы.

Экземпляр 9 (Произведение есть Λ -множество). Внутреннее произведение $\Pi' X Y$ само является Λ -множеством, в котором терм t реализует семейство f , если для каждого α и каждого терма u в расширенном контексте, реализующего α , надлежащим образом переименованный и применённый терм реализует $f(\alpha)$.

2. К системам типов, способных к самореинтерпретации

Мы обосновываем, что системы типов должны уметь переинтерпретировать собственные выражения в произвольной внутренней модели самих себя: вооружившись единственным инструментом переинтерпретации, можно выразить широкий круг свойств, таких как полиморфность вселенных, внешняя параметричность, возможность вести синтетическую математику внутри системы и многое другое. В этих целях мы исследуем пределы самореинтерпретации и представляем две системы типов, обладающие этим свойством, а именно HM-R и MLTT-R, доказываем их базовые свойства и приводим несколько примеров полезных внутренних моделей.

2.1 Введение

Рассмотрим следующую известную функцию, проходящую по элементам списка, написанную на языке Agda:

```
map : ∀ {A B : Set} → (A → B) → List A → List B
map f [] = []
map f (x :: xs) = f x :: map f xs
```

Наслаживаясь и комбинированностью этой функции, и простотой её типа, мы бы хотели как можно больше переиспользовать её определение. До определённого уровня мы, безусловно, смогли бы это сделать: в конце концов, `map` полиморфна по исходному и целевому типам, поэтому мы можем инстанцировать её произвольными типами. Однако, если бы мы нашли волшебную лампу с джином теории типов внутри, мы бы попросили его исполнить нам три желания:

1. Переиспользовать `map` на более высоких уровнях вселенных, например, по *операторам типов*, применить *проходку по списку типов*:

$$(\text{Set} \rightarrow \text{Set}) \rightarrow \text{List Set} \rightarrow \text{List Set}$$

2. Автоматически получить доказательство свободной теоремы для `map` – то есть некоторое выражение типа

$$\begin{aligned} &\forall A A' (Q : A \rightarrow A' \rightarrow \text{Prop}), \forall B B' (R : B \rightarrow B' \rightarrow \text{Prop}), \\ &\quad \forall f f', (\forall x x', Q x x' \rightarrow R(fx) (f'x')) \rightarrow \\ &\quad \forall (xs : \text{List } A)(xs' : \text{List } A'), Q^* xs xs' \rightarrow R^*(\text{map } f xs)(\text{map } f' xs'); \end{aligned}$$

3. По данным группоидов A и B и функтора $f : A \rightarrow B$ построить функтор $\text{map } f$ между группоидами $\text{List } A$ и $\text{List } B$.

Первое желание, разумеется, исполняется путём корректировки определения и придания ему полиморфности уровней. Хотя теорема из второго желания уже справедлива в системах с параметричностью, получение доказательства *внутри* системы без участия человека возможно лишь в тех системах типов, где параметричность *internalizada* [5, 4]. Третье желание на данный момент полагается на ручную работу (или на использование метапрограммирования [18]), хотя и с парой индукций по списку: одна для определения группоида List и ещё несколько для доказательства функториальности map .

Мы предлагаем иной подход. Прежде чем выправлять определение map , вспомним всю метатеоретическую мощь, которой уже обладает наша система типов. Упомянутая выше параметричность полиморфных определений – первый результат, который приходит на ум; отметим, что она достигается за счёт задания *реляционного толкования* суждения о типизированности [21]. И вправду, некоторые из самых интересных метатеоретических результатов выражаются путём построения определённой *модели* системы типов: синтетические результаты в теории гомотопий основаны на $((\infty, 1)$ -)группоидной модели (гомотопической) теории типов [11, 3]; использование Макбрайдом смещения для полиморфности вселенных [15, 16] можно рассматривать как построение модели системы типов в более высоких вселенных самой себя и последующую интерпретацию смещённых конструкций в ней. Тогда все наши перечисленные выше желания могли бы быть исполнены путём построения подходящей модели системы типов *внутри* неё самой (отсюда мы называем их *внутренними моделями*) и последующую инстанциацию map в ней. На протяжении всей статьи мы будем называть системы типов с возможностью самореинтерпретации *рефлексивными*.

2.1.1 Вклад

- Наша главная цель – построение реализуемой рефлексивной системы зависимых типов. Основная трудность возникает из определённой теоремы о невозможности, обсуждённой в разделе 2.2, которая заставляет нас ограничить мощность переинтерпретации, чтобы проверка типов оставалась решаемой.
- Для удобства изложения мы сначала предлагаем рефлексивную систему типов на основе системы типов Хиндли–Милнера, называемую НМ-R (раздел 2.3). Мы доказываем, что НМ-R корректна, консервативна относительно просто типизированного лямбда-исчисления, и что глобальный вывод типов в ней решаем (раздел 2.4).
- Опираясь на наработанный при разработке НМ-R опыт, мы наконец готовы погрузиться в рефлексивную версию теории типов Мартина–Лёфа, обозначенную как MLTT-R (раздел 2.5). Мы доказываем корректность и консервативность MLTT-R (раздел 2.5.3).
- Как и задумано, благодаря переинтерпретации MLTT-R включает в себя как ограниченную форму полиморфности вселенных, так и внешнюю параметричность. И, конечно, MLTT-R можно использовать как систему для синтетической математики с внутренним переиспользованием синтетических результатов (раздел 2.5.4).

2.2 Теорема о невозможности внутренней интерпретации

Итак, мы хотим построить рефлексивную систему типов. То есть такую систему, в которой пользователь может как определить внутреннюю модель системы, так и применить

её к корректно типизированным выражениям, чтобы получить их внутренние интерпретации. Естественно, мы бы хотели, чтобы пользовательские модели уважали тождественные равенства системы; в системе зависимых типов это можно естественно обеспечить путём использования типа тождества для записи соответствующих равенств внутри модели. Было бы хорошо, если бы перед переинтерпретацией выражения система сначала приводила его к нормальной форме, чтобы избавиться от артефактов построения в результате интерпретации. Наконец, в системе зависимых типов естественно ожидать сокращения открытых выражений, поэтому мы хотим, чтобы наша интерпретация вычислялась на открытых выражениях.

Доказываемая ниже теорема утверждает, что все три свойства одновременно несоместимы в одной системе.

Сначала пара определений. Мы работаем в классической типовой структуре, которая вводит вид контекстов Γ , выражения t, t' и типы T , связанные соотношением типизированности $\Gamma \vdash t : T$; вычисление задаётся одношаговым отношением сокращения $t \rightsquigarrow t'$ на выражениях.

Определение 48 (Наивное вычислительное переинтерпретирование). *Оператор типов $F[_]$ называется наивным вычислительным переинтерпретированием типов, если он уважает вычисления на типах (то есть если существует отношение сокращения на типах ($\rightsquigarrow$) и $T \rightsquigarrow T'$, то $F[T] \rightsquigarrow^* F[T']$) и вычисляется как примитивно рекурсивная функция на нормальных формах типов. Аналогично, оператор выражений с дыркой $\llbracket _ \rrbracket$ называется наивным вычислительным переинтерпретированием выражений, если он уважает вычисления на выражениях, вычисляется как примитивно рекурсивная функция на нормальных формах выражений и, если $\Gamma \vdash t : T$, то $\Gamma \vdash \llbracket t \rrbracket : F[T]$.*

Теорема 7 (Теорема о невозможности наивного переинтерпретирования). *В конфлюэнтной системе типов с типом функций и наивным вычислительным переинтерпретированием, если сокращение коммутрует с подстановкой, то для произвольных корректно типизированных s и t существует и такой, что:*

$$\text{app}[\lambda x.t][s] \rightsquigarrow^* u \leftarrow^* \llbracket t[x := s] \rrbracket$$

где app – это действие $\llbracket _ \rrbracket$ на применениях.

Следовательно, $\text{app}[\lambda x.t][s]$ и $\llbracket t[x := s] \rrbracket$ тождественно эквивалентны, что заставляет β -сокращения в модели держаться тождественно, а не предикативно, что прямо противоречит желаемому результату. Аналогичные теоремы доказуемы для любого другого сокращения, присутствующего в системе; иными словами, **предикативного равенства недостаточно**. Доказательство ведётся прямым примером:

Доказательство. Даны корректно типизированные $\Gamma \vdash s : S$ и $\Gamma, x : S \vdash t : T$. Рассмотрим следующее выражение:

$$\Gamma, x : S \rightarrow T, y : S \vdash [xy] : F[T]$$

Выражение под интерпретацией находится в нормальной форме, поэтому всё выражение вычисляется до $\text{app}[\llbracket x \rrbracket][\llbracket y \rrbracket]$. Подставляя $\lambda x.t$ вместо x и s вместо y , получаем $\text{app}[\lambda x.t][s]$.

С другой стороны, если сначала выполнить подстановку, то получим $\Gamma \vdash \llbracket (\lambda x.t)s \rrbracket : F[T]$. Так как наивное переинтерпретирование уважает сокращение, это сокращается до $\llbracket t[x := s] \rrbracket$.

Поскольку сокращение коммутует с подстановкой и система конфлюэнтна, должно существовать такое u , до которого оба выражения сокращаются. $\square$

Фундаментально, приведённое выше доказательство эксплуатирует открытость исходного выражения; если ограничить переинтерпретирование вычислением только на замкнутых выражениях, мы можем получить корректную систему без требования тождественных равенств в моделях, как это иллюстрируется нашей системой без зависимых типов НМ-Р,

описанной в разделе 2.3. Для системы зависимых типов, однако, наша теорема о невозможности оставляет нам три варианта:

1. Ограничить модели теми, в которых равенства выполняются тождественно, а не предикативно.
2. Не выполнять переинтерпретирование, пока выражение не замкнуто. Чтобы рассуждать о открытых выражениях, уравнения, описывающие поведение интерпретации, представляются в виде предикативных аксиом.
3. Вообще не сокращать открытые выражения внутри переинтерпретирования, выполняя переинтерпретацию как есть. Чтобы упростить результат переинтерпретации, можно переписать его с использованием равенств, присутствующих в модели.

Если взглянуть на существующие работы о смещении [15, 16], внутренней параметричности [4] и отображённой теории типов [13] через призму переинтерпретирования внутри фиксированной модели (или семейства моделей), то видно, что соответствующие модели действительно валидируют равенства тождественно; поэтому, по-видимому, следует выбрать первый подход. Он также наиболее удовлетворительный с вычислительной точки зрения. Однако если вы хотите корректную систему, в которой модели могут быть определены пользователем, необходимо ограничивать модели через дисциплину типизации. Следуя известным на данный момент методам [6], ограничение моделей таким образом требовало бы определения типа моделей как типа алгебр индуктивно-индуктивно-рекурсивного типа (а для эквивалентных теорий типов даже индуктивно-индуктивно-рекурсивного симплициального типа), что существенно выходит за рамки данной статьи.

Из двух оставшихся вариантов последний лучше подходит для практических систем зависимых типов, поскольку он корректно вычисляет на открытых выражениях. Кроме того, он не ограничивает класс возможных моделей, одновременно предоставляя преимущества первого подхода: если в модели уравнения выполняются тождественно, переписывание результата переинтерпретации для его сокращения не требуется. Единственный недостаток состоит в том, что сокращения *после* интерпретации, возможно, более затратны, чем те, которые можно было бы выполнить *до* неё.

2.3 Почти просто типизированный почти рефлексивный язык: HM-R

Мы начнём с более простого языка, чтобы проработать детали и разобраться, как именно следует переинтерпретировать выражения внутренним образом, прежде чем переходить к системе зависимых типов. Простейший язык, для которого мы знаем много интересных моделей, – это просто типизированное лямбда-исчисление с произведениями, которое мы в дальнейшем будем обозначать как “STLC”; в этом разделе мы представим язык, способный интерпретировать выражения STLC как конструкции в моделях STLC, которые могут быть определены в нашем языке.

Итак, какие же модели STLC? Помимо обычной множества-теоретической семантики, где $S \rightarrow T$ интерпретируется как функция из S в T , а $S \times T$ – как декартово произведение, существует доказательственная семантика, которая соотносит выражения типа T с интуиционистскими доказательствами предложения $\llbracket T \rrbracket$, где $(\rightarrow)$ теперь понимается как импликация, а $(\times)$ – как конъюнкция. Это, конечно, классическое соответствие Карри–Ховарда [12]; как множества-теоретическая, так и доказательственная семантика обобщаются полным описанием моделей STLC, данным Йоахимом Ламбеком [14]. Согласно этому *соответствию*

Карри–Ховарда–Ламбека, моделями STLC являются в точности картезианские замкнутые категории (или КЗК для краткости). Определения варьируются; мы будем использовать определение 49.

Определение 49. *Картезианская замкнутая категория (КЗК) – это категория $\mathcal{C}$ с терминальным объектом 1 , бинарными продуктами (обозначаемыми $\times$) и экспонентами (обозначаемыми $\Rightarrow$), то есть: для любых объектов A и B существуют объекты $A \times B$ и $A \Rightarrow B$ вместе с соответствующими морфизмами:*

$$\begin{aligned} \pi_1 : A \times B &\rightarrow A, & \pi_2 : A \times B &\rightarrow B, \\ \langle f, g \rangle : C &\rightarrow A \times B \quad \text{для } f : C \rightarrow A, g : C \rightarrow B, \\ \text{ev} : (A \Rightarrow B) \times A &\rightarrow B, \\ \lambda f : C \Rightarrow B &\rightarrow (C \Rightarrow B) \quad \text{для } f : C \times B \rightarrow B, \end{aligned}$$

удовлетворяющих стандартным равенствам КЗК.

2.3.1 Синтаксис и семантика HM-R

Синтаксис HM-R расширяет синтаксис STLC на оператор переинтерпретации `interp`. Конкретнее, типизированные выражения $\Gamma \vdash t : T$ задаются по индукции следующей грамматикой, где x – переменные, Γ, Γ' – контексты, A, B – типы (для краткости мы отождествляем типы с выражениями и используем $A : \mathbb{T}$ для обозначения $\text{Type} \vdash A : \mathbb{T}$):

```

expr  ::= id
        | lambda id . expr
        | expr expr
        | pair expr expr
        | expr .1
        | expr .2
        | interp expr expr expr
ctx    ::= empty
        | id : type , ctx
type   ::= id
        | type .1
        | type .2
        | type -> type
        | type x type
id     ::= x

```

Здесь `interp  $e_1$   $e_2$   $e_3$` означает «интерпретацию e_3 в модели e_1 , в контексте e_2 ». Правила типизации (в BNF-подобной нотации, где $\Gamma; \Gamma' \vdash t : T$ означает $\Gamma \vdash \text{interp } e_1 \ e_2 \ t : T'$) задаются следующим образом:

Здесь **Model** – тип моделей STLC (КЗК), **Ctx** m – тип контекстов в модели m , а последняя посылка $e_1 \vdash e_2 \vdash t : T'$ означает, что t корректно типизировано в модели e_1 в контексте e_2 с типом T' .

2.3.2 Определение модели STLC внутри HM-R

Чтобы сделать переинтерпретацию полезной, мы должны дать конкретную модель. Рассмотрим тривиальную модель, в которой все типы интерпретируются как единый объект $\star$, все функции – как единственный морфизм $\star \rightarrow \star$, и аналогично для остальных конструкций.

Формально, пусть `Const` обозначает тип константных КЗК. Определение `Const` в HM-R выглядит следующим образом:

```
Const : Model
Const = record
  { Ob = Star
    ; Mor = Unit
    ; id = unit
    ; comp = const
    ; prod = const
    ; exp = const
    ; ev = unit
    ; lam = const
    ; pi1 = unit
    ; pi2 = unit
    ; pair = const
  }
  where
    Star : Type
    Star = Unit
```

Эта модель, конечно, не несёт полезной информации, но она демонстрирует механизм.

2.3.3 Полиморфность вселенных через переинтерпретацию

Ключевое наблюдение состоит в том, что если у нас есть модель M системы типов, мы можем применить переинтерпретацию к *типам*, получая таким образом *тип в модели*. Это позволяет нам «поднимать» типы на один уровень вселенной: если A – тип во вселенной Set_0 , то $\text{interp } M \sqcup A$ становится объектом в модели M , который, в свою очередь, является типом во вселенной Set_1 .

Таким образом, полиморфность вселенных достигается не как первичное свойство системы типов, а как *следствие* наличия переинтерпретации.

2.3.4 Параметричность через переинтерпретацию

Аналогично, для получения параметричности достаточно рассмотреть реляционную модель: вместо того чтобы интерпретировать типы как множества, мы интерпретируем их как пары множеств, связанных отношением. Тогда полиморфная функция автоматически удовлетворяет теореме о свободной теореме [21, 5].

В HM-R это можно выразить следующим образом:

```
RelModel : Model
RelModel = record
  { Ob =  $\forall A \rightarrow A \times A$ 
    ; Mor =  $\forall \{A\ B\}$ 
      → (A → A → Rel)
      → (B → B → Rel)
      → (A → B → Rel)
      → (B → A → Rel)
      → (A → B → Rel)
```

```
; ...
}
```

(Полное определение приводится в приложении.)

2.4 Свойства НМ-Р

2.4.1 Корректность

Теорема 8 (Корректность НМ-Р). *Каждое выражение, для которого выведено суждение о типизированности $\Gamma \vdash t : T$, действительно имеет тип T в любой модели НМ-Р.*

Доказательство. Доказательство ведётся по индукции на вывод суждения о типизированности. Базовые случаи очевидны. Для случая лямбда-абстракции используем индукционное предположение и определение экспоненты в КЗК. Случай применения сводится к композиции морфизмов. Случай **interp** следует из гипотезы о том, что e_1 является корректной моделью и t корректно типизировано в ней. $\square$

2.4.2 Консервативность

Теорема 9 (Консервативность над STLC). *Если выражение t типизировано в НМ-Р с типом T , и t и T не содержат оператора **interp**, то t типизировано в STLC с тем же типом T .*

Доказательство. Прямая индукция на вывод: правило для **interp** не участвует в выводе, поскольку ни t , ни T не содержат **interp**. Остальные правила совпадают с правилами STLC. $\square$

2.4.3 Вывод типов

Теорема 10 (Решаемость вывода типов). *Глобальный вывод типов в НМ-Р решаем.*

Доказательство. Алгоритм вывода сводится к унификации типов (для случаев STLC) и проверке корректности модели (для случаев **interp**). Унификация типов в STLC решаем. Проверка корректности модели сводится к проверке равенств морфизмов, что решаем для конечных КЗК. $\square$

2.5 Рефлексивная теория типов Мартина–Лёфа: MLTT-R

Система НМ-Р является хорошей отправной точкой, но она лишена зависимых типов, продуктов Σ , и других ключевых конструкций, нужных для серьёзной математики. В этом разделе мы переносим идеи НМ-Р в контекст теории типов Мартина–Лёфа.

2.5.1 Синтаксис

Синтаксис MLTT-R расширяет синтаксис STLC следующими конструкциями:

- **Зависимые продукты** (Π-типы): $\Pi x : A. B$ с правилом введения $\lambda x : t$ (где t имеет тип B при $x : A$) и элиминации $(\lambda x : t) a$.
- **Зависимые суммы** (Σ-типы): $\Sigma x : A. B$ с правилом введения (a, b) и элиминации через **res**.

- Тип тождества $\text{Id}_A(a, b)$ с правилом введения refl и элиминации через J-правило.
- Уровни вселенных $\text{Set}_0, \text{Set}_1, \dots$ с правилом $\text{Set}_i : \text{Set}_{i+1}$.

Кроме того, мы сохраняем оператор interp , адаптированный для зависимых типов:

$$\text{interp } e_1 \ e_2 \ t$$

где e_1 – модель MLTT, e_2 – контекст в этой модели, а t – выражение, которое мы хотим переинтерпретировать.

2.5.2 Модели MLTT

Моделью MLTT-R является категория с семействами (КсФ, или CwF), оборудованная необходимыми конструкциями: закрытая сумма (для Σ -типов), эквивалентность (для типа тождества), и закрытая экспонента (для Π -типов).

Формальное определение КсФ дано Altenkirch и Reus

Определение 50 (CwF). Категория с семействами – это структура $(\text{Ctx}, \text{Ty}, \text{El}, \text{Sub})$, где:

- Ctx – тип контекстов;
- $\text{Ty}(\Gamma)$ – тип типов в контексте Γ ;
- $\text{El}(\Gamma, A)$ – тип выражений типа A в контексте Γ ;
- $\text{Sub}(\Gamma, \Delta)$ – тип подстановок из Γ в Δ .

вместе с соответствующими операциями и аксиомами.

2.5.3 Свойства MLTT-R

Теорема 11 (Корректность MLTT-R). Каждое выражение, для которого выведено суждение о типизированности, действительно корректно типизировано в любой модели MLTT-R.

Доказательство. Доказательство ведётся по индукции на вывод. Случаи, не содержащие interp , сводятся к стандартной корректности MLTT в КсФ. Случай interp следует из гипотезы о корректности модели и корректности выражения в ней. $\square$

Теорема 12 (Консервативность над MLTT). Если выражение t типизировано в MLTT-R с типом T , и t и T не содержат interp , то t типизировано в MLTT с тем же типом T .

Доказательство. Аналогично доказательству консервативности HM-R над STLC: правило для interp не участвует в выводе, поскольку ни t , ни T не содержат interp . $\square$

2.5.4 Применения переинтерпретации

Полиморфность вселенных

Рассмотрим функцию map из введения. В стандартной MLTT она имеет тип:

$$\forall A B : \text{Set}_0. (A \rightarrow B) \rightarrow \text{List } A \rightarrow \text{List } B$$

Чтобы применить её к типам (т.е. к объектам Set_0), мы используем переинтерпретацию в константной модели:

$\text{interp Const} \parallel \text{map}$

Это даёт нам функцию типа:

$$(\text{Set}_0 \rightarrow \text{Set}_0) \rightarrow \text{List Set}_0 \rightarrow \text{List Set}_0$$

которая применяет оператор типов к списку типов.

Параметричность

Используя реляционную модель (раздел), мы можем автоматически получать свободные теоремы для полиморфных определений. Например, теорема о композиции map :

$$\text{map } g (\text{map } f \text{ } xs) = \text{map } (g \circ f) \text{ } xs$$

следует из параметричности map в реляционной модели.

Синтетическая математика

Синтетическая математика позволяет вести математические рассуждения внутри системы типов. С переинтерпретацией мы можем не только вести эти рассуждения, но и *переиспользовать* их на более высоких уровнях вселенных.

Например, если мы доказали в MLTT-R теорему о существовании и единственности терминального объекта в группоиде, мы можем переинтерпретировать это доказательство в модели, где группоиды заменяются на 2-группоиды, получая аналогичную теорему для 2-группоидов без повторного доказательства.

2.6 Сравнение с существующими подходами

2.6.1 Смещение Макбрайда

Политика «смещения» Макбрайда [15, 16] достигает полиморфности вселенных путём введения семейств типов, зависящих от «уровня смещения». Это можно рассматривать как построение модели системы типов в более высоких вселенных самой себя и последующую интерпретацию смещённых конструкций в ней.

В отличие от нашего подхода, смещение Макбрайда является *встроенной* механикой системы типов: все выражения автоматически «знают» о своём уровне смещения. В MLTT-R же переинтерпретация является явной: пользователь сам решает, когда и в какой модели переинтерпретировать выражение. Это придаёт MLTT-R большую гибкость ценой дополнительной явности.

2.6.2 Внутренняя параметричность

Подход Altenkirch и Bernardy к внутренней параметричности [5, 4] достигает параметричности путём введения отношений как первоклассных граждан системы типов. Это позволяет выражать свободные теоремы внутри системы.

Наш подход через переинтерпретацию в реляционной модели достигает аналогичного результата, но без изменения основной системы типов. Параметричность становится *наблюдаемым* свойством, а не *встроенным* механизмом.

2.6.3 Отображённая теория типов

Отображённая теория типов [13] обобщает подход смещения Макбрайда, используя язык «отображений» для описания семейств типов на различных уровнях. Это даёт более общую рамау для обсуждения полиморфности вселенных и параметричности.

Наш подход комплиментарен этим работам: вместо изменения языка типов, мы добавляем оператор переинтерпретации, который позволяет «смотреть» на выражения через линзу произвольной модели.

2.7 Обсуждение и ограничения

2.7.1 Ограничения на модели

Как обсуждалось в разделе 2.2, наши модели не обязаны удовлетворять равенствам тождественно; достаточно предикативных равенств. Однако это означает, что результат переинтерпретации может требовать дополнительного сокращения для приведения к нормальной форме. В практических системах это может привести к потере производительности.

2.7.2 Вывод типов

Вывод типов в MLTT-R сложнее, чем в HM-R, из-за зависимых типов и типа тождества. Однако правило для `interp` не усложняет вывод типов: оно требует лишь проверки того, что выражение корректно типизировано в модели, что является отдельной (и объёмной) проверкой, не зависящей от окружающего вывода.

2.7.3 Возможности для будущих работ

- **Автоматическое нахождение моделей.** Мы можем исследовать, можно ли автоматически на основе типа выражения подбирать подходящую модель для переинтерпретации.
- **Метапрограммирование.** Переинтерпретация может служить основой для метапрограммирования: вместо того чтобы писать код генератора кода, мы переинтерпретируем выражение в модели, где «выполнение» означает «генерацию кода».
- **Связь с доказательством проверкой.** Можно исследовать, как переинтерпретация соотносится с подходом «проверка вместо вывода» особенно в контексте двунаправленной проверки типов
- **Модальные системы.** Переинтерпретация может быть обобщена на модальные системы типов, где «модальность» задаёт способ переинтерпретации

2.8 Заключение

Мы показали, что система типов может самореинтерпретироваться: определять модели самой себя внутри себя и переинтерпретировать свои выражения в этих моделях. Мы представили две системы типов, HM-R и MLTT-R, доказали их корректность и консервативность, и продемонстрировали ряд применений, включая полиморфность вселенных, параметричность и синтетическую математику.

Наш подход отличается от существующих работ о смещении, внутренней параметричности и отображённой теории типов тем, что переинтерпретация является явной операцией, а не встроенным механизмом. Это придаёт системе большую гибкость: пользователь может выбрать произвольную модель для переинтерпретации, а не ограничиваться «стандартной» семантикой, встроенной в систему.

В будущих работах мы намерены расширить MLTT-R до полной теории гомотопий, исследовать связь переинтерпретации с модальными системами типов и разработать эффективные алгоритмы вывода типов для реальных реализаций.

Заключение

Основные результаты исследования

TODO

С акцентом на изложение полученных научных результатов (а не на пересказ содержания диссертации)

Список выносимых на защиту результатов TODO

Общие выводы

TODO

(без повторения основных результатов)

Перспективы дальнейшей разработки

TODO

Основные публикации соискателя

TODO

А. Список сокращений и условных обозначений

В. Словарь терминов

С. Список литературы

- [1] National Security Agency. *Software Memory Safety*. Апр. 2023. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF.
- [2] Thorsten Altenkirch и Nils Anders Danielsson. «Termination Checking in the Presence of Nested Inductive and Coinductive Types». В: *Partiality and Recursion in Interactive Theorem Provers, PAR@ITP 2010, Edinburgh, UK, July 15, 2010*. Под ред. Ekaterina Komendantskaya, Ana Bove и Milad Niqui. Т. 5. EPiC Series. EasyChair, 2010, с. 101—106. DOI: 10.29007/N51D.
- [3] Thorsten Altenkirch и Ondrej Rypacek. «A Syntactical Approach to Weak omega-Groupoids». В: *Computer Science Logic (CSL'12) - 26th International Workshop/21st Annual Conference of the EACSL*. Под ред. Patrick Cégielski и Arnaud Durand. Т. 16. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2012, с. 16—30. ISBN: 978-3-939897-42-2. DOI: 10.4230/LIPIcs.CSL.2012.16. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CSL.2012.16>.
- [4] Thorsten Altenkirch и др. «Internal Parametricity, without an Interval». В: *Proc. ACM Program. Lang.* 8.POPL (янв. 2024). DOI: 10.1145/3632920.
- [5] Jean-Philippe Bernardy, Patrik Jansson и Ross Paterson. «Parametricity and dependent types». В: *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming*. ICFP '10. Baltimore, Maryland, USA: Association for Computing Machinery, 2010, с. 345—356. ISBN: 9781605587943. DOI: 10.1145/1863543.1863592.
- [6] Marc Bezem и др. *A Note on Generalized Algebraic Theories and Categories with Families*. 2021. arXiv: 2012.08370 [math.CT]. URL: <https://arxiv.org/abs/2012.08370>.
- [7] Mario Carneiro. «The Type Theory of Lean». Дис. ... маг. Carnegie Mellon University, апр. 2019.
- [8] Randal Clouston и др. *Programming and Reasoning with Guarded Recursion for Coinductive Types*. 2015. arXiv: 1501.02925 [cs.PL].
- [9] Thierry Coquand и Gérard Huet. «The calculus of constructions». В: *Information and Computation* 76.2 (1988), с. 95—120. ISSN: 0890-5401. DOI: [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3).
- [10] Edsger W. Dijkstra. «The humble programmer». В: *Commun. ACM* 15.10 (окт. 1972), с. 859—866. ISSN: 0001-0782. DOI: 10.1145/355604.361591.
- [11] Martin Hofmann и Thomas Streicher. «The groupoid model refutes uniqueness of identity proofs». В: *Proceedings of the Ninth Annual IEEE Symp. on Logic in Computer Science, LICS 1994*. Под ред. Samson Abramsky. Paris, France: IEEE Computer Society Press, июль 1994, с. 208—212.

-
- [12] W. Howard. «The Formulae-as-Types Notion of Construction». В: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Под ред. J. Seldin и J. Hindley. Academic Press, 1980, с. 479—490.
 - [13] Astra Kolomatskaia и Michael Shulman. «Displayed type theory and semi-simplicial types». В: *Mathematical Structures in Computer Science* 35 (2025), e34. DOI: 10.1017/S096012952510025
 - [14] Joachim Lambek. «From λ -calculus to cartesian closed categories». В: *To H. B. Curry: Essays on Combinatory Logic, Lambda-Calculus and Formalism*. Под ред. J. P. Seldin и J. R. Hindley. London: Academic Press, 1980, с. 375—402.
 - [15] Conor McBride. *Crude but Effective Stratification*. 2011. URL: <https://mazzo.li/epilogue/index.html%3Fp=857&cpage=1.html> (дата обр. 09.07.2026).
 - [16] Conor McBride. *Crude but Effective Stratification*. 2012. URL: <https://personal.cis.strath.ac.uk/conor.mcbride/Crude.pdf> (дата обр. 09.07.2026).
 - [17] Leonardo de Moura и Sebastian Ullrich. «The Lean 4 Theorem Prover and Programming Language». В: *Automated Deduction – CADE 28*. Под ред. André Platzer и Geoff Sutcliffe. Cham: Springer International Publishing, 2021, с. 625—635. ISBN: 978-3-030-79876-5.
 - [18] Wojciech Nawrocki и др. «A Certifying Proof Assistant for Synthetic Mathematics in Lean». В: *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP '26. Rennes, France: Association for Computing Machinery, 2026, с. 88—103. ISBN: 9798400723414. DOI: 10.1145/3779031.3779087.
 - [19] Chris Newcombe и др. «How Amazon web services uses formal methods». В: *Commun. ACM* 58.4 (март 2015), с. 66—73. ISSN: 0001-0782. DOI: 10.1145/2699417.
 - [20] Ulf Norell. «Towards a practical programming language based on dependent type theory». Дис. . . . док. SE-412 96 Göteborg, Sweden: Department of Computer Science и Engineering, Chalmers University of Technology, сент. 2007.
 - [21] John C Reynolds. «Types, abstraction and parametric polymorphism». В: *Information Processing 83*. Под ред. R. E. A. Mason. North-Holland. 1983, с. 513—523.
 - [22] Peiyang Song, Kaiyu Yang и Anima Anandkumar. *Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean*. 2025. arXiv: 2404.12534 [cs.AI].
 - [23] The Coq Development Team. *The Coq Proof Assistant*. Вер. 8.20. Сент. 2024. DOI: 10.5281/zenodo.14542673.
 - [24] Andrea Vezzosi, Anders Mörtberg и Andreas Abel. «Cubical agda: a dependently typed programming language with univalence and higher inductive types». В: *Proc. ACM Program. Lang.* 3.ICFP (июль 2019). DOI: 10.1145/3341691.
 - [25] Philip Wadler. «Propositions as types». В: *Commun. ACM* 58.12 (нояб. 2015), с. 75—84. ISSN: 0001-0782. DOI: 10.1145/2699407.
 - [26] Б. Пирс. *Типы в языках программирования: пер. с англ.* Лямбда пресс, 2012. ISBN: 9785791300829.

D. Список иллюстрированного материала

Е. Приложения